

can you design diagrams? like a software architecture diagram?

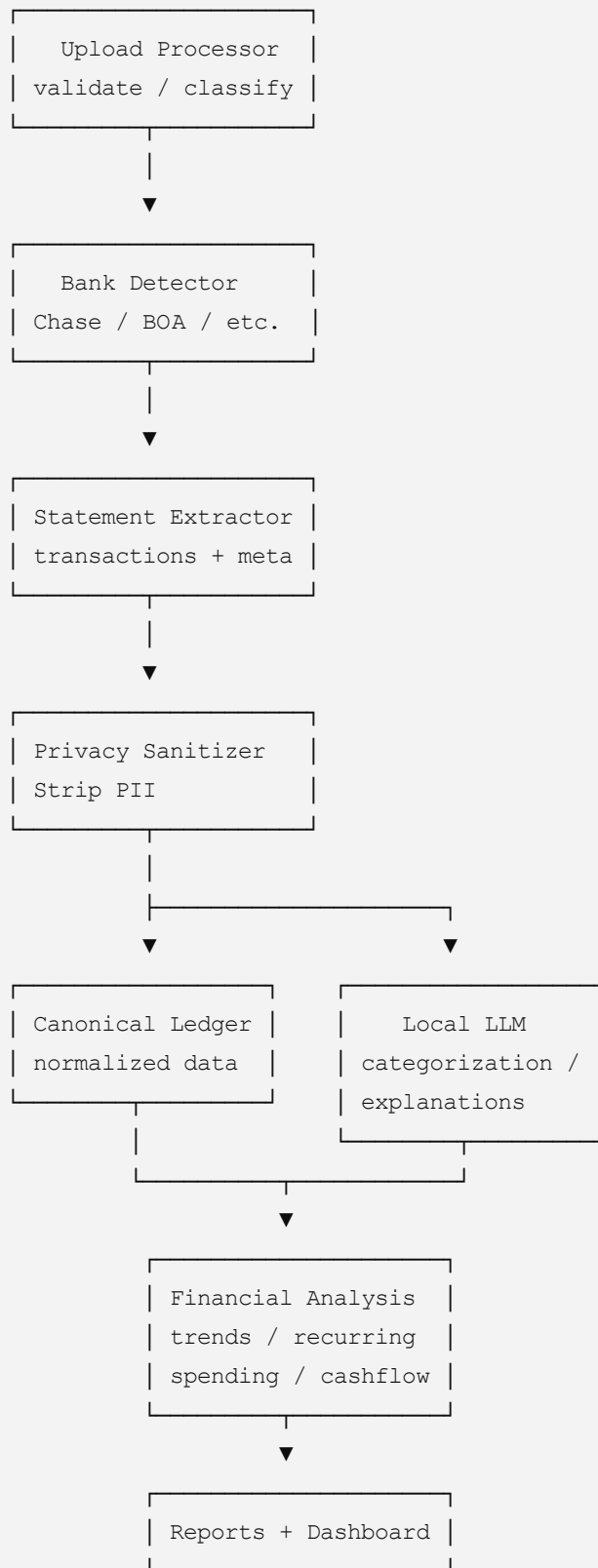
Yes. I can design **software architecture diagrams**, not just describe them.

I can help with diagrams such as:

- **High-level system architecture** — frontend → API → services → databases → external systems
- **AWS architecture** — API Gateway, Lambda, S3, DynamoDB, Cognito, SQS/EventBridge, CloudWatch, etc.
- **Data-flow diagrams** — what happens from user action through processing/storage/output
- **Component diagrams** — responsibilities and boundaries between modules
- **Trust/security boundaries** — browser, backend, private data, third-party APIs, LLMs
- **AI/RAG architectures** — ingestion → chunking → embeddings → vector DB → retrieval → LLM
- **Sequence diagrams** — step-by-step interactions during a workflow
- **Deployment architecture** — local/dev/staging/prod, CI/CD, cloud services
- **Failure-path diagrams** — what happens when extraction, APIs, models, databases, etc. fail

For example, your **Bank Statement Analyzer** could start conceptually like:

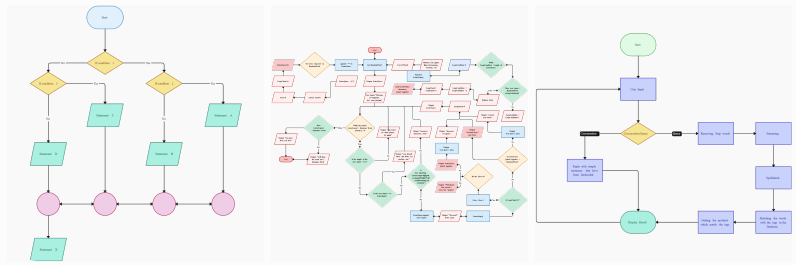




But that is just a rough text representation. I can also create a **proper visual architecture diagram** with grouped components, arrows, trust boundaries, labels, and professional styling suitable for your portfolio.

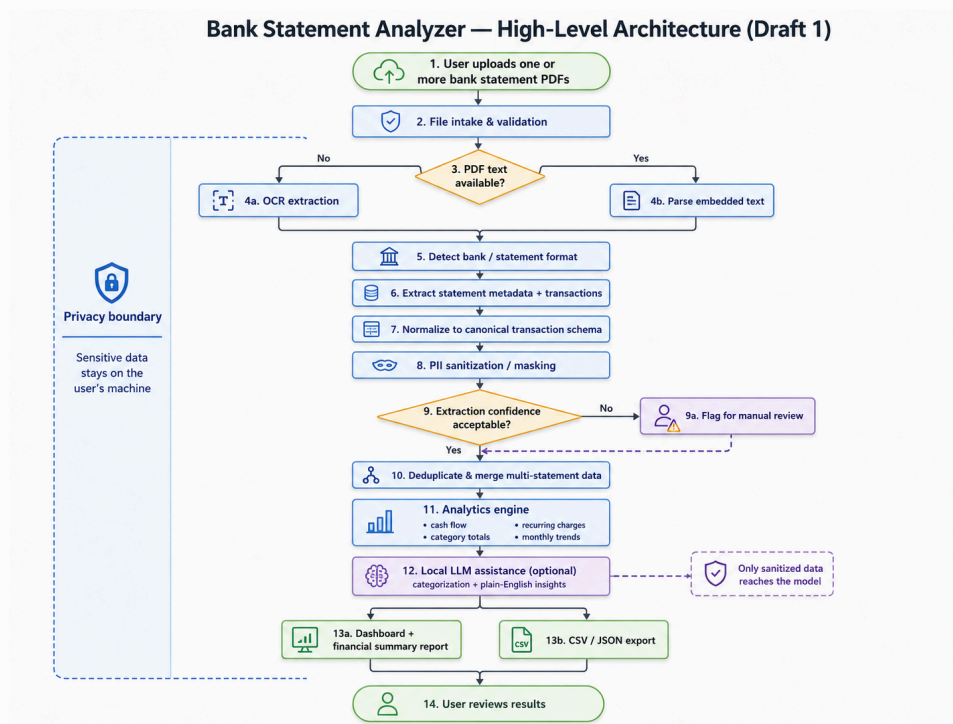
More importantly, I wouldn't jump straight to making a pretty diagram. We should first make sure the **architecture itself is defensible**—responsibilities, data flow, privacy boundaries, failure behavior, and why each component exists. Then the diagram becomes documentation of the design instead of decoration.

Today 11:12 PM



i definitely want the diagram for the Bank Statement Analyzer. I want to design it with you step by step. i found this link <https://www.programiz.com/article/flowchart-programming> or the pasted images from here <https://creately.com/guides/flowchart-examples-programming/>

Worked for 1m 10s



that is a good diagram. however, you miss the part where i told you i wanted to design it with you step by step so I can learn and enforce the decision behind the project

You're right. I jumped to the artifact instead of using the diagram as a teaching tool.

The better process is: **we design the architecture together, one decision at a time, and only draw each section after the reasoning is settled.** That matters because otherwise

you end up with a diagram you can show, but not defend.

The first generated diagram is still useful as a rough destination, but treat it as **Draft 0**, not the architecture.

We should build it in layers:

1. **User journey / happy path**
2. **Input boundary**
3. **Statement detection**
4. **Extraction strategy**
5. **Canonical data model**
6. **Privacy boundary**
7. **Validation + confidence**
8. **Multi-statement merging**
9. **Analytics**
10. **LLM boundary**
11. **Output/reporting**
12. **Failure paths**
13. **Deployment/runtime boundaries**

And for every layer, we force three things:

- **What problem is this component solving?**
- **Why does it exist as its own responsibility?**
- **What breaks if we remove or combine it?**

Step 1 — Define the system flow before architecture

Forget technologies for now.

The system starts with this user action:

A user selects multiple bank statements from one or more banks and asks the application to analyze their finances over a period of time.

The first question is therefore not "Should we use OCR?"

It is:

What should happen immediately after the user selects the statements?

My proposed first flow is:

START

User selects statements



System validates files



System determines which files it can process



Valid statements enter processing pipeline

Notice what I deliberately did **not** include yet:

OCR

LLM

Postgres

Python

Bank detection

PII removal

Analytics

Those are implementation decisions. We're still defining behavior.

Decision 1: What does "valid statement" mean?

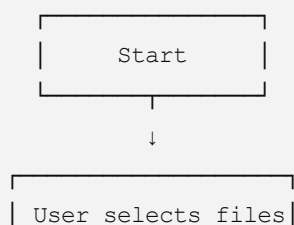
For your project, I would define an accepted input initially as:

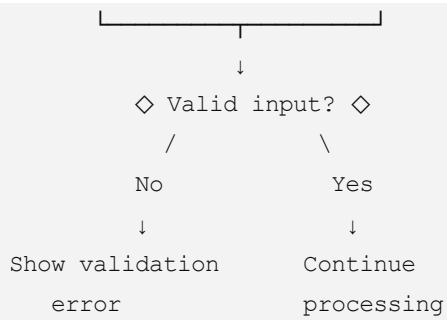
- ✓ PDF
- ✓ readable
- ✓ appears to be a bank statement
- ✓ supported size/page count
- ✓ not password protected

Rejected input:

- ✗ image of random document
- ✗ corrupted PDF
- ✗ encrypted/password-protected statement
- ✗ unsupported file type
- ✗ document we cannot confidently identify as a bank statement

That gives us our first real decision node:





This looks simple, but it encodes an important architectural rule:

Bad input must be rejected before expensive or sensitive processing begins.

That's the first invariant I'd put in the project documentation.

Why this matters

Suppose we skip validation and immediately send everything to an extractor.

Then:

```
random.pdf
  ↓
OCR
  ↓
bank detector
  ↓
LLM
  ↓
failure
```

You've wasted compute and made the privacy boundary harder to reason about.

Instead:

```
random.pdf
  ↓
validation
  ↓
STOP
```

Much cleaner.

Your first architecture decision

For **v1**, I recommend this:

The analyzer accepts PDF bank statements only. Images, CSV imports, screenshots, OFX/QFX, and direct bank connections are explicitly out of scope.

That constraint is important. Trying to support every financial input format immediately would destroy the project scope.

So the first piece of our architecture becomes:

```
[User]
  ↓
[Select one or more PDF statements]
  ↓
<Valid supported PDF?>
  ├── No → [Reject + explain why]
  └── Yes → [Statement Processing Pipeline]
```

That's all I would design today for **Step 1**.

The next step should be much more interesting:

Once we accept a PDF, how do we determine whether it contains embedded text, requires OCR, and which bank produced it?

That decision determines the entire extraction architecture.

I am okay with step 1 so far. what is a statement failed to upload? where do we handle that?

That belongs **before validation**.

Right now our Step 1 is missing a small but important distinction:

```
User selects files
  ↓
Upload / file transfer succeeds?
  ├── No → show upload error
  └── Yes → validate file
```

A failed upload means the application never successfully received the file into the processing boundary. Examples: browser/network interruption, file read failure, request too large, timeout, client-side abort, or backend refusing the payload before validation begins.

So the corrected Step 1 is:

```
START
  ↓
User selects one or more PDFs
```

```

↓
Attempt file upload / intake
↓
<Upload successful?>
├─ No
│   ↓
│   Mark file as failed
│   ↓
│   Show reason + allow retry/remove
└─ Yes
    ↓
    <Valid supported PDF?>
    ├─ No → Reject + explain why
    └─ Yes → Continue processing

```

The distinction matters:

Upload failure

"We did not successfully receive the file."

Validation failure

"We received the file, but we refuse to process it."

Those are different failure classes and should stay separate.

For multiple statements, I would also avoid making this all-or-nothing. If the user uploads 10 statements and 1 fails:

```

9 uploaded successfully
1 failed

→ process the 9 valid files
→ clearly flag the failed one
→ let the user retry it

```

Do **not** throw away nine good statements because one upload failed.

That gives us another architectural invariant:

A failure affecting one statement must not automatically invalidate unrelated statements in the same batch.

This will matter later when we design extraction failures too. A Bank of America statement failing OCR shouldn't necessarily destroy successful Chase and Capital One results.

So I'd revise our first stage into three separate responsibilities:

```
File Selection
  ↓
File Intake / Upload
  ↓
File Validation
  ↓
Statement Processing
```

And that's already telling us something useful architecturally: **upload/intake and validation should not be treated as the same thing.**

okay. i think step 1 is all set.

Good. Step 1 is locked.

The final Step 1 flow is:

```
START
  ↓
User selects one or more PDF statements
  ↓
Attempt file intake/upload
  ↓
<Upload successful?>
  |— No
  |   ↓
  |   Mark that file as failed
  |   ↓
  |   Show reason + retry/remove
  |— Yes
      ↓
  <Valid supported PDF?>
    |— No → Reject + explain why
    |— Yes → Continue processing
```

And we now have two architectural rules from it:

- **Upload failure and validation failure are separate concerns.**
- **One failed statement should not automatically invalidate the rest of the batch.**

Next is **Step 2: Statement extraction path** — specifically, deciding how the system handles text-based PDFs versus scanned PDFs, and when OCR should be used. That is where the architecture starts getting more interesting.

there should be a check to detect if ocr or not

Exactly. That check belongs at the start of Step 2, before we choose an extraction path.

The key distinction is:

Does this PDF already contain usable embedded text, or is it effectively an image that requires OCR?

So Step 2 should begin like this:

```
Valid PDF
  ↓
Inspect PDF content
  ↓
<Usable embedded text found?>
  ├── Yes
  │   ↓
  │   Parse embedded text
  └── No
      ↓
      Run OCR
      ↓
      Extract text
```

The important word there is **usable**.

A PDF can technically contain text but still be garbage for extraction. For example:

```
Expected:
Checking Account
Beginning Balance: $4,291.10
08/05/2026  AMAZON  -45.82

Bad embedded extraction:
C h e c k i n g  A c c o u n t
B e g i n n i n g  ...
4 2 9 1  ...
```

So I would **not** make the decision:

Does PDF contain text?

Too weak.

Make it:

Can we extract enough usable text from the PDF?

That means the detector can eventually consider things like:

- extracted character count
- percentage of pages with text
- whether expected text structures exist
- whether output is mostly unreadable characters
- whether table/text layout is recoverable

For v1, we don't need an overly clever ML detector. The architectural responsibility is simply:

PDF Inspection

```
↓
<Text extraction viable?>
├─ Yes → Native PDF extraction
└─ No  → OCR extraction
```

And this gives us another strong architectural invariant:

OCR is a fallback, not the default extraction method.

That's important because OCR is slower, more computationally expensive, and generally introduces more extraction errors than reading actual embedded PDF text.

There's also a failure path we should already acknowledge:

Run OCR

```
↓
<OCR successful?>
├─ No → Flag statement as extraction failure
└─ Yes → Continue
```

So Step 2 is starting to look like:

Valid supported PDF

```
↓
Inspect PDF
↓
<Usable embedded text?>
/      \
Yes      No
↓        ↓
Native text  OCR
extraction  extraction
↓          ↓
└────────┘
```

↓
Extracted text

I would stop there for this decision.

We should **not yet decide how we detect the bank**, because first we've established the contract for the next stage:

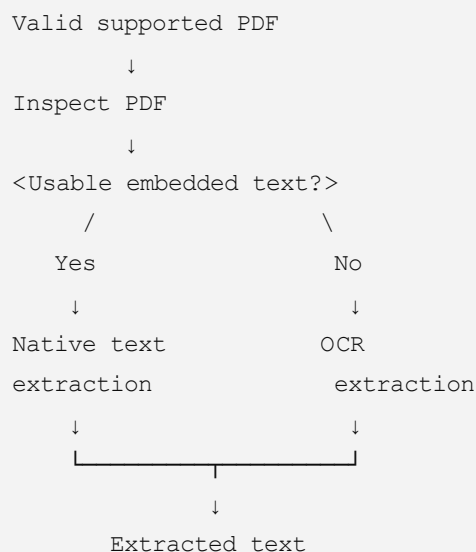
Regardless of whether the source was native PDF text or OCR, the next component receives extracted text.

That separation will make the architecture cleaner later.

ok

Good. Step 2 is locked.

We now have this pipeline:



And the rule behind it is:

OCR is a fallback path, not the default path.

Next should be **Step 3: identify the bank and statement format from the extracted content**. That decision is important because it determines whether we use bank-specific parsers, generic extraction, or a hybrid approach.

should we do that yet? if the user import 2 years of statements for at least 5 banks. that's a lot of statements.

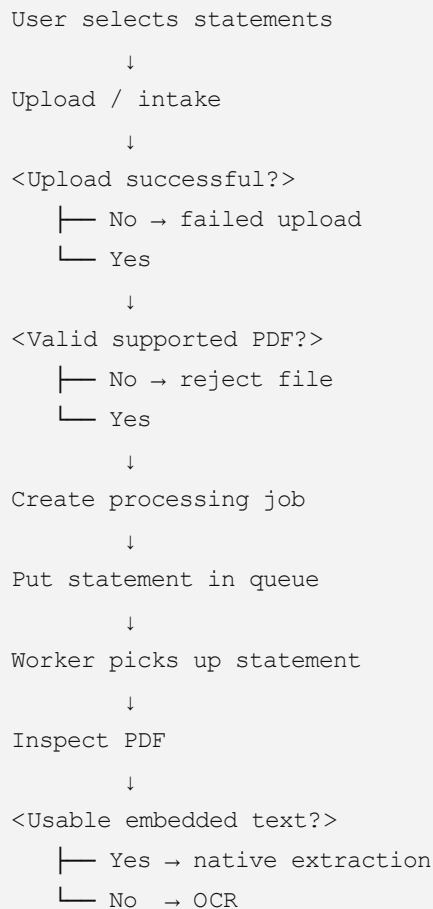
shouldn't we put them in a queue for processing

Yes. That is the better architecture.

If the user uploads 2 years of statements across 5 banks, processing them synchronously one-by-one in the request path would be a bad design. The browser should not sit there waiting while dozens of PDFs get parsed and OCR runs.

The queue belongs **between successful validation and expensive statement processing**.

So I would revise the flow to:



That gives us a much stronger system boundary.

The key reason is simple:

Upload is user-facing. Extraction is background work.

Those should not be tightly coupled.

With a queue, 120 statements can become 120 independent jobs. That gives you:

- controlled concurrency
- retries per statement

- failure isolation
- progress tracking
- ability to pause/resume later
- no long-running browser request
- no need to re-upload successful files if one statement fails

The batch itself can also have a parent job:

```
Analysis Batch
├─ Statement Job 1
├─ Statement Job 2
├─ Statement Job 3
├─ ...
└─ Statement Job 120
```

Each statement gets its own status:

```
UPLOADED
VALIDATED
QUEUED
PROCESSING
EXTRACTED
FAILED
COMPLETED
```

And the whole batch can report:

```
82 / 120 completed
31 processing
5 queued
2 failed
```

This also gives us another architectural invariant:

Each statement is processed independently, while the batch coordinates overall progress and final aggregation.

I would lock that in before we move to bank detection.

One nuance: I would not queue invalid files. Validation should happen first, because there is no point filling the queue with work we already know we should reject.

So the order should be:

```
Upload
↓
Validate
```

```
↓
Queue
↓
Process
```

not:

```
Upload
↓
Queue
↓
Validate
```

The next architecture decision should now be: **what exactly goes into the queue: the PDF bytes themselves, or a reference to where the PDF was temporarily stored?** That choice affects memory, privacy, retry behavior, and scalability.

you are going to have to help me think here. i don't know yet

That's exactly where I should help. Don't guess. We can reason from the requirements.

The decision is:

When a statement is queued, should the queue contain the whole PDF, or just a reference to the PDF?

For this project, I recommend:

Store the PDF temporarily, and put only a file reference + metadata in the queue.

So conceptually:

```
User uploads PDF
↓
Temporary file storage
↓
Create statement job
↓
Queue message contains:
- job_id
- batch_id
- file_location
- filename
- upload timestamp
↓
Worker retrieves PDF
```

↓
Process statement

Why this is better: queues are for **work instructions**, not large documents. If you shove entire PDFs into queue messages, you create size limits, memory pressure, retry headaches, and unnecessary duplication. A small pointer is cleaner.

Think of it like a restaurant ticket system. The kitchen ticket should say:

Order #482, table 6, chicken pasta

It should not physically contain the ingredients.

For your analyzer:

```
Queue = "Process statement 81"  
Storage = actual statement 81
```

There's also a privacy consequence. Since the statements contain sensitive financial data, the temporary storage needs a clear lifecycle:

```
Upload  
↓  
Temporary encrypted storage  
↓  
Processing  
↓  
Analysis completed  
↓  
Delete original statement
```

Unless the user explicitly chooses to keep it.

That gives us another architectural rule:

Raw bank statements are temporary processing artifacts, not permanent application data.

I think that rule fits the privacy goal of this project very well.

Now there is one important fork we should think through next: **local app vs hosted app changes what “temporary storage” means.**

For a local app:

User's disk
↓

```
Temporary application directory
  ↓
Worker
  ↓
Delete
```

For a hosted version:

```
Browser
  ↓
Private object storage
  ↓
Worker
  ↓
Delete
```

But architecturally, both can use the same abstraction:

```
File Storage
```

We don't need to choose S3, local filesystem, MinIO, or anything else yet.

So I would add this between validation and the queue:

```
<Valid PDF?>
  ↓ Yes
Store temporarily
  ↓
Create Statement Job
  ↓
Queue job reference
```

Then the worker begins Step 2:

```
Worker receives job
  ↓
Retrieve statement
  ↓
Inspect PDF
  ↓
<Usable embedded text?>
```

That is a much cleaner architecture. And now you should be able to defend why the queue does not carry the PDF itself.

Good. Lock that decision in.

At this point, the architecture has four important rules:

- Upload/intake and validation are separate.
- Invalid files never enter the processing queue.
- Each statement becomes its own background job.
- The queue carries a **reference to the temporary file**, not the PDF itself.

So our current flow is:

```
User selects PDFs
      ↓
Upload / intake
      ↓
<Upload successful?>
  ├── No → mark failed + retry/remove
  └── Yes
      ↓
<Valid supported PDF?>
  ├── No → reject + explain
  └── Yes
      ↓
Store PDF temporarily
      ↓
Create statement job
      ↓
Queue file reference + job metadata
      ↓
Worker receives job
      ↓
Retrieve PDF
      ↓
Inspect PDF
      ↓
<Usable embedded text?>
  ├── Yes → native text extraction
  └── No  → OCR extraction
      ↓
Extracted text
```

The next decision I would tackle is **not bank detection yet**.

Before that, we should decide what happens if the worker fails while processing a statement.

For example:

```
OCR crashes
PDF parser throws an exception
worker process dies
temporary file is missing
processing times out
```

That forces us to define **retry behavior and failure states**, which is important before we build more logic on top of the queue.

we can retry 2 or 3 times on failures. but how important is it to retry also. it is just pdfs that is available to re-upload.

Retry is useful, but it should be **selective**, not automatic for every failure.

For your project, I'd use this rule:

Retry only failures that are likely temporary. Do not retry failures that are deterministic.

Examples:

- Worker crashed → retry
- Temporary file read error → retry
- OCR process timed out → retry
- Transient storage issue → retry
- PDF is corrupted → do not retry
- Password-protected PDF → do not retry
- Unsupported format → do not retry
- OCR completed but text is unusable every time → probably do not keep retrying

So instead of “retry every failed statement 3 times,” use:

```
Processing failed
  ↓
<Retryable failure?>
  | Yes
  |   ↓
  |   Retry up to 2-3 times
  |   ↓
  |   Still failing?
  |     | Yes → mark FAILED
  |     | No  → continue
  |   No
  ↓
Mark FAILED
```

For this project, **2 retries after the first attempt** is enough. That means max 3 total attempts.

Why not more? Because the user still owns the original PDFs. If a statement genuinely cannot be processed, the app can say:

“This statement could not be processed. Retry or re-upload it.”

That’s simpler than building aggressive recovery logic.

The architectural point is this:

Retries exist to recover from infrastructure/transient failures, not to hide bad input.

And because the source PDF is temporary but still available during the batch, the worker can retry without requiring the user to upload it again.

So I’d add one more job state:

```
QUEUED
PROCESSING
RETRYING
COMPLETED
FAILED
```

That is enough for v1. Don’t over-engineer dead-letter queues and complex backoff policies yet. The design should prove you understand them, but the project doesn’t need enterprise-scale machinery for local bank statements.

agreed and okay to move on to the next

Next is **bank and statement-format detection**.

At this point, the worker has a PDF and has already chosen native text extraction or OCR. Now we have extracted text. The question becomes:

How does the system know which bank produced the statement, and which parser should handle it?

I recommend a **hybrid detector**, not an LLM-first detector.

```
Extracted text
↓
Bank / format detector
```

```
↓
<Recognized format?>
├─ Yes → use bank-specific parser
└─ No  → use generic parser / flag unsupported
```

Why bank-specific parsers? Because statements from Chase, Bank of America, Capital One, etc. do not expose transaction data in exactly the same layout. A generic parser will eventually become fragile.

But we should not hard-code detection from filename alone. A user can rename:

```
statement.pdf
checking-july.pdf
bank1.pdf
```

Detection should primarily inspect **statement content**. For example, the detector could look for known markers:

```
"JPMorgan Chase Bank"
"Bank of America"
"Capital One"
"Wells Fargo"
```

plus layout/version clues.

I would design the output of this stage as something like:

```
bank = "chase"
statement_type = "checking"
parser_version = "chase_checking_v1"
confidence = 0.97
```

That `parser_version` matters more than it looks. A bank can redesign its statements later. You don't want:

```
Chase → one parser forever
```

You want:

```
Chase
├─ checking_v1
├─ checking_v2
└─ credit_card_v1
```

So our architectural rule becomes:

Detect the statement format from extracted content before parsing transactions, and version parsers independently of the bank name.

One failure path also matters:

```
Extracted text
  ↓
Detect format
  ↓
<Supported format?>
├─ Yes → parse
└─ No  → mark as unsupported / manual review
```

Do **not** silently guess the closest bank parser. That could create believable but incorrect financial data, which is worse than failing loudly.

So I would lock Step 3 as:

```
Extracted text
  ↓
Detect bank + statement format
  ↓
<Supported with sufficient confidence?>
├─ No → unsupported/manual review
└─ Yes
      ↓
Select versioned parser
      ↓
Continue to structured extraction
```

The next step after this should be **what exactly the parser extracts and what the canonical transaction schema should look like**. That is one of the most important design decisions in the whole project.

okay

Good. Step 3 is locked.

The current processing path is now:

```
Extracted text
  ↓
Detect bank + statement format
  ↓
<Supported with sufficient confidence?>
├─ No → unsupported / manual review
```

```
└─ Yes
    ↓
Select versioned parser
    ↓
Continue to structured extraction
```

Next we should define the **canonical data model** before we keep adding components.

That means deciding what every parser must output, regardless of whether the source is Chase, Bank of America, Capital One, or another bank.

The reason is simple:

Bank-specific parsing should stop at the boundary where data becomes normalized.

After that point, the analytics engine should not care which bank the transaction came from.

So the next design question is:

What fields must every normalized transaction contain?

That is the right next step.

okay continue

Next is the canonical transaction schema. This is one of the most important parts because every bank-specific parser should end here.

The rule is:

Different banks can have different statement layouts, but they must all produce the same internal transaction shape.

For v1, I would start with these fields:

```
Transaction
├─ transaction_id
├─ statement_id
├─ account_id
├─ transaction_date
├─ posted_date
├─ description_raw
├─ description_normalized
├─ amount
├─ direction
├─ balance_after
```

```
| category
| source_bank
| source_statement_type
| extraction_confidence
└ source_page
```

A few of these need explanation.

`description_raw` preserves exactly what the statement gave us.

Example:

```
POS PURCHASE 08/15 STARBUCKS 1234 BOSTON MA
```

`description_normalized` is the cleaned version:

```
Starbucks
```

Do not overwrite the raw description. You may need it later for debugging, auditability, or improving parsers.

`amount` should be stored as a positive numeric value, and `direction` should tell us whether it is money in or money out:

```
amount = 45.82
direction = DEBIT
```

Instead of relying on:

```
-45.82
+1200.00
```

Both approaches work, but explicit direction is easier to reason about across banks because statement conventions differ.

`balance_after` should be optional. Some statement layouts provide running balance; others may not.

`category` should also be optional at extraction time. Parsing and categorization are different responsibilities.

So initially:

```
category = null
```

Later the categorization engine can assign:

```
category = "Dining"
```

And `extraction_confidence` is important because OCR or parsing can be imperfect.

Example:

```
extraction_confidence = 0.96
```

That lets later stages decide whether a transaction needs review.

The parser contract then becomes:

```
Bank-specific statement
    ↓
Bank-specific parser
    ↓
Canonical Statement
    ↓
Canonical Transactions[]
```

For example:

```
Chase parser
    ↓
Transaction {
  date: 2026-08-15
  description_raw: "POS PURCHASE STARBUCKS..."
  amount: 6.45
  direction: DEBIT
}
```

and:

```
Bank of America parser
    ↓
Transaction {
  date: 2026-08-15
  description_raw: "STARBUCKS STORE 123..."
  amount: 6.45
  direction: DEBIT
}
```

After normalization, the analytics engine should not care which parser produced them.

I would also define a separate canonical statement object:

```
Statement
├─ statement_id
├─ batch_id
├─ bank
├─ account_type
├─ account_identifier_masked
├─ statement_start_date
├─ statement_end_date
├─ opening_balance
├─ closing_balance
├─ currency
├─ parser_version
└─ extraction_status
```

That gives you this hierarchy:

```
Batch
├─ Statement
│   ├── Transaction
│   ├── Transaction
│   └── Transaction
├─ Statement
│   └── ...
```

The key invariant here is:

Analytics must operate on canonical data, never directly on bank-specific parser output.

That prevents the rest of the application from turning into:

```
if bank == "chase":
...
elif bank == "boa":
...
elif bank == "capital_one":
...

```

which would become a maintenance mess.

One design choice I would not lock yet is `account_id`. We should decide whether accounts are derived automatically from statement metadata or whether the user confirms account grouping. That matters when someone uploads two checking accounts from the same bank.

ok

Good. The next decision is **account identity and grouping**.

This matters because the same user may upload:

```
Chase checking — account ending 1234
Chase checking — account ending 9876
Capital One credit card — ending 4444
```

We should not group statements only by bank name.

I recommend this rule:

Derive a stable internal `account_id` from statement metadata, but never store the full account number.

Conceptually:

```
Statement metadata
  ↓
bank
account type
masked account digits
  ↓
Account grouping logic
  ↓
internal account_id
```

Example:

```
bank = "Chase"
account_type = "checking"
account_last4 = "1234"
```

becomes something like:

```
account_id = acct_7f31...
```

The user never needs to see that internal ID. In the UI they see:

```
Chase Checking •••• 1234
```

The important part is that later statements for the same account should resolve to the same internal account.

So:

June Chase ••••1234
July Chase ••••1234
August Chase ••••1234

should all map to:

Account A

But:

Chase ••••9876

must become:

Account B

I would also add a safety valve:

```
<Account match confident?>  
├─ Yes → attach to existing account  
└─ No  → create provisional account / ask for confirmation
```

We probably won't need frequent user confirmation if the bank + account type + masked digits are available, but we should not silently merge ambiguous accounts.

That gives us another invariant:

Statements are grouped by account identity, not merely by bank or statement type.

After this, the next major step is **validation of extracted financial data**. Before analytics trusts the transactions, we need to check whether the parser output makes financial sense. That is where opening balance, closing balance, credits, debits, and transaction totals become useful.

ok.

Next we should design **financial consistency validation**.

This is different from the earlier file validation.

Earlier validation asked:

“Is this a file we can process?”

Now we're asking:

“Did we extract the statement correctly enough to trust the numbers?”

That distinction matters.

Suppose a parser gives us:

```
Opening balance:    $2,000
Total deposits:     $1,500
Total withdrawals:  $900
Closing balance:     $2,100
```

That is internally consistent:

```
2000 + 1500 - 900 = 2600
```

Actually, no — it is **not** consistent. The expected closing balance would be `$2,600`.

That means something probably went wrong:

- a transaction was missed
- an amount was parsed incorrectly
- the opening/closing balance was wrong
- the statement contains fees or adjustments we didn't extract
- OCR damaged a value

This is exactly why validation must happen **before analytics**.

Step 5 — Validate extracted statement data

Our flow becomes:

```
Canonical Statement
    +
Canonical Transactions[]
    ↓
Financial Validation
    ↓
<Checks pass?>
  └─ Yes → trusted canonical data
  └─ No → low confidence / manual review
```

The first validation rule I would implement is:

```
opening_balance
+ total_credits
- total_debits
≈ closing_balance
```

The `≈` matters because some bank formats or rounding situations may create tiny differences.

For example:

```
Opening balance      $3,200.00
Deposits             $2,400.00
Withdrawals         -$1,350.00
Fees                 -$15.00
-----
Expected closing     $4,235.00
```

If the statement says:

```
Closing balance      $4,235.00
```

great.

If it says:

```
$2,435.00
```

the parser should not quietly continue.

But we shouldn't rely on only one check

I would divide validation into three levels.

1. Structural checks

Did we extract the things the parser claims should exist?

```
statement_start_date exists
statement_end_date exists
bank identified
account identified
transactions found
currency identified
```

Not every field has to be mandatory, but required fields should exist.

2. Transaction checks

For every transaction:

```
date is valid
amount is numeric
direction is valid
transaction is within reasonable statement period
description isn't empty
```

For example:

```
transaction_date = 2097-14-76
```

clearly indicates extraction failure.

3. Financial reconciliation

Check whether totals make sense.

```
Opening
+ Credits
- Debits
- Fees
± Adjustments
≈ Closing
```

This is probably your strongest quality check.

We should not make validation simply PASS / FAIL

I'd use a validation result with confidence/severity.

Conceptually:

```
validation_status =
    VALID
    WARNING
    FAILED
```

Example:

```
VALID
```

Everything reconciles.

WARNING

One transaction has a low OCR confidence score, but the statement totals reconcile.

FAILED

Closing balance is off by \$4,200 and several transactions could not be parsed.

That's far more useful than treating every imperfection equally.

This also changes our confidence model

Earlier we talked about:

```
extraction_confidence = 0.96
```

I would now separate two ideas:

```
extraction_confidence
```

“How confident are we that the text/fields were extracted correctly?”

and:

```
validation_status
```

“Does the resulting financial data actually make sense?”

A parser can be technically confident and still produce financially inconsistent data.

For example:

```
OCR confidence: 99%
```

but it confidently read:

```
$8,000
```

instead of:

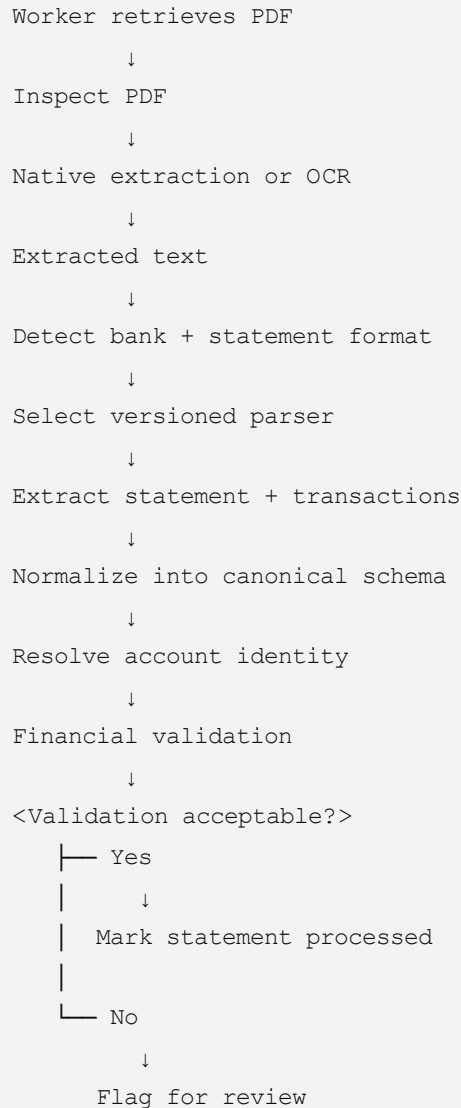
```
$800.00
```

The reconciliation check can catch that.

That separation is important.

Where it fits

Our worker pipeline now looks roughly like this:



And I would establish this invariant:

No statement enters portfolio-level financial analytics until its normalized data has passed minimum validation.

That's a strong design decision because otherwise one badly parsed PDF could corrupt an entire two-year financial report.

There is one subtle question we should handle next:

If the user uploads 120 statements and **117 pass validation while 3 fail**, should we let the overall analysis run using the 117 good statements?

I think the answer should be **yes, but with very clear incomplete-data warnings**. That's the next decision worth designing.

ok

Then we should lock that behavior in:

A small number of failed statements should not block analysis of the successful ones.

So for a batch:

```
120 statements uploaded
```

```
117 VALID
```

```
2 WARNING
```

```
1 FAILED
```

The system can still generate the report, but it must clearly say the analysis is incomplete.

Something like:

```
Analysis coverage:
```

```
117 of 120 statements successfully processed
```

```
Excluded:
```

```
- Chase Checking - March 2025
```

```
- Capital One - July 2025
```

```
- Bank of America - October 2024
```

The important part is that we do **not** silently pretend the dataset is complete.

That gives us another architectural invariant:

Partial analysis is allowed, but missing or excluded statements must be visible to the user and reflected in report confidence.

This also means the batch itself needs a status beyond just `COMPLETED` or `FAILED`. I'd use:

```
PROCESSING
```

```
COMPLETED
```

```
COMPLETED_WITH_WARNINGS
```

```
FAILED
```

So if 117/120 succeeded:

```
COMPLETED_WITH_WARNINGS
```

Now the pipeline can move into the next major phase:

```
Validated canonical statements
      ↓
Batch aggregation
      ↓
Cross-statement analytics
```

That is where we finally start looking across months and accounts instead of treating every PDF independently. This is also where duplicate detection, recurring expenses, monthly trends, cash-flow summaries, and account-level rollups belong.

ok

Next is **batch aggregation and cross-statement analytics**.

At this point, individual statements have already been extracted, normalized, grouped to accounts, and validated. Now we can safely ask questions across the full dataset.

The flow becomes:

```
Validated statements
      ↓
Aggregate by account + time period
      ↓
Deduplicate overlapping transactions
      ↓
Build unified transaction history
      ↓
Run analytics
```

The first thing to solve is **deduplication**.

Why? Because users may accidentally upload the same statement twice, or overlapping exports may contain the same transaction more than once.

A naive system could double-count:

```
08/15 Starbucks    $6.45
08/15 Starbucks    $6.45
```

So we need a rule like:

Transactions that strongly match on account, date, amount, and normalized description should be treated as possible duplicates.

Not automatically deleted with zero evidence. Better:

<Likely duplicate?>

- |— High confidence → collapse duplicate
- |— Medium confidence → flag
- |— No → keep both

Then we can build a clean ledger.

After that, analytics can operate on:

Unified Ledger

```
|— Account A
|   |— Jan
|   |— Feb
|   |— ...
|— Account B
|— Account C
```

For v1, I'd keep analytics focused on the actual value proposition:

```
Monthly income
Monthly spending
Net cash flow
Category totals
Recurring charges
Merchant totals
Largest expenses
Spending trends
Account-level summaries
```

And one architecture rule matters a lot here:

Analytics must be deterministic first.

Meaning these should come from code/math, not from an LLM.

For example:

```
monthly_spend = sum(debits)
net_cash_flow = credits - debits
```

Not:

LLM, please estimate my spending this month

The LLM should sit **after** the analytics engine and explain the results, not calculate the financial truth.

So the next architecture section should look like:

```
Validated canonical data
    ↓
Deduplication
    ↓
Unified ledger
    ↓
Deterministic analytics engine
    ↓
Structured financial insights
```

That gives us another invariant:

Financial calculations come from deterministic application logic; the LLM may explain results but does not own the numbers.

That is a very strong design choice for this project.

ok. for duplication, we can look at transaction info and date+time stamps for accuracy

Yes, with one correction: **bank statements often do not include transaction timestamps**. Many only have a transaction date and/or posted date.

So deduplication should use the strongest combination of fields available, not depend on time.

I'd design it like this:

```
Candidate Transaction
    ↓
Compare against transactions
from the same account
    ↓
Match signals:
- transaction date
- posted date
- timestamp, if available
- amount
```

- debit/credit direction
- normalized merchant/description
- reference/transaction ID, if available

↓

Calculate duplicate confidence

For example, these are almost certainly duplicates:

Statement A:

08/14/2026 | STARBUCKS 1234 | \$7.82 | DEBIT

Statement B:

08/14/2026 | STARBUCKS STORE 1234 | \$7.82 | DEBIT

But this is dangerous:

08/14/2026 | STARBUCKS | \$7.82

08/14/2026 | STARBUCKS | \$7.82

The user could genuinely have gone to Starbucks twice. Same merchant + same amount + same date is **not enough evidence to automatically delete one**.

So I would use three outcomes:

Duplicate confidence

↓

HIGH
Same account + transaction ID
or extremely strong field match
→ automatically collapse
POSSIBLE
Same date + amount + merchant
but insufficient unique evidence
→ keep + flag
NOT DUPLICATE
→ keep both

There is another useful signal we haven't discussed: **statement overlap**.

Suppose someone accidentally uploads:

Chase January 2026.pdf

Chase January 2026-copy.pdf

Before comparing hundreds of individual transactions, we can recognize:

```
same account
same statement start date
same statement end date
same opening balance
same closing balance
```

That strongly suggests the **entire statement** is duplicated.

So I would actually give the system two layers:

```
Validated statements
    ↓
Statement-level duplicate detection
    ↓
Transaction-level duplicate detection
    ↓
Unified ledger
```

That's cleaner and cheaper.

Decision I'd lock in

Deduplication uses multiple deterministic signals. Transactions are automatically removed only when duplicate confidence is high; ambiguous matches are preserved and flagged rather than silently discarded.

That's especially important for financial software. **A false positive—deleting a real \$500 transaction—is worse than temporarily showing a possible duplicate.**

And we still keep the source relationship:

```
transaction
    ↓
source_statement_id
source_page
```

So if the system says something is duplicated, we can always explain **where both copies came from**.

That gives us a nice auditability rule too:

Every normalized transaction must remain traceable to its source statement.

I would lock this into the aggregation design before we move on.

Next is **analytics design**—but we should still keep the LLM out of the core math.

At this stage, we have a trusted unified ledger. Now we decide what the analytics engine should produce deterministically.

I'd structure the first version around five outputs:

```
Unified Ledger
  ↓
Analytics Engine
  ↓
├─ Cash flow
├─ Spending breakdown
├─ Recurring charges
├─ Merchant analysis
└─ Trend analysis
```

For example, cash flow:

```
monthly_income
monthly_spending
net_cash_flow = income - spending
```

Spending breakdown:

```
Housing
Food
Transport
Subscriptions
Utilities
Entertainment
Other
```

Recurring charges should not be based on a simple “same amount every month” rule. That would miss real subscriptions where the amount changes slightly.

We should match on things like:

```
normalized merchant
similar amount range
repeat interval
same account
transaction direction
```

So:

```
Netflix
Jan 15    $22.99
Feb 15    $22.99
Mar 15    $22.99
```

is easy.

But this can also be recurring:

```
Electric Company
Jan 8      $142.18
Feb 7      $166.42
Mar 9      $151.73
```

Same merchant, similar monthly cadence, variable amount.

That means recurring detection is its own deterministic component:

```
Unified Ledger
    ↓
Recurring Pattern Detector
    ↓
Recurring Charge Candidates
```

Then merchant analysis can answer:

```
How much did I spend at Amazon over 12 months?
How often did I use Uber?
Which merchants increased the most?
```

Trend analysis can compare periods:

```
Jan-Jun average dining: $480/month
Jul-Dec average dining: $690/month

increase = 43.8%
```

The important architectural rule is:

Analytics outputs structured facts first; explanations come later.

So instead of sending raw transactions directly to an LLM, the system should produce something like:

JSON

```
{
  "monthly_spending": {
    "2026-01": 4210.55,
    "2026-02": 3988.20
  },
  "top_merchants": [
    {"merchant": "Amazon", "total": 2310.44},
    {"merchant": "Uber", "total": 1802.17}
  ],
  "recurring_charges": [
    {"merchant": "Netflix", "frequency": "monthly"}
  ]
}
```

Then later the LLM can say:

Your spending increased mainly because transportation and dining rose over the last four months.

But the LLM is explaining data that already exists.

I'd lock this invariant:

The analytics engine owns financial facts; the LLM owns interpretation and natural-language explanation.

Next, we should decide **where categorization belongs**, because that affects both recurring detection and spending analysis.

ok

Next is **transaction categorization**.

I would not make the LLM the primary categorizer either.

Use a layered approach:

```
Normalized transaction
  ↓
Rule / merchant mapping
  ↓
<Category known?>
  └─ Yes → assign category
  └─ No
    ↓
  Local model / LLM assistance
```

↓

Why this order?

Because obvious cases should be deterministic:

```
NETFLIX → Entertainment / Subscription  
SHELL → Fuel  
COMCAST → Utilities
```

You do not need an LLM call every time you see the same merchant.

Also, merchant normalization happens before categorization:

```
"UBER *TRIP"  
"UBER TRIP"  
"UBER TECHNOLOGIES"  
    ↓  
Normalized merchant: Uber  
    ↓  
Category: Transportation
```

That gives us three separate responsibilities:

```
Raw description  
    ↓  
Merchant normalization  
    ↓  
Category assignment  
    ↓  
Confidence
```

And the result could look like:

```
merchant_normalized = "Uber"  
category = "Transportation"  
category_source = "rule"  
category_confidence = 1.0
```

Or:

```
merchant_normalized = "XYZ SERVICES"  
category = "Home Services"  
category_source = "local_model"  
category_confidence = 0.72
```

If confidence is low, we should not pretend certainty:

<Confidence acceptable?>

- └ Yes → use category
- └ No → mark Uncategorized / review

I would also allow the user to correct a category later:

```
XYZ SERVICES
System: Shopping
User: Home Maintenance
```

That correction should become useful for future transactions from the same normalized merchant.

So another architectural invariant:

User corrections override automated categorization.

And:

Categorization should degrade to `Uncategorized`, not to a confident guess.

The flow now becomes:

```
Unified Ledger
  ↓
Merchant normalization
  ↓
Deterministic categorization
  ↓
Fallback local model assistance
  ↓
Categorized Ledger
  ↓
Analytics
```

That order matters because category-based analytics should consume already-categorized transactions.

Next we should design the **LLM privacy boundary**—exactly what data the model is allowed to see, what gets stripped first, and what the model is forbidden from doing.

ok

Next is the **LLM privacy boundary**.

This is one of the most important parts of the project because your stated goal is that sensitive financial information should not be exposed unnecessarily.

I'd define the rule first:

The LLM never receives raw bank statements or unfiltered extracted text.

That immediately gives us a boundary:

```
Raw PDF
  ↓
Extraction
  ↓
Normalization
  ↓
PII sanitization
  ↓
Structured financial data
  ↓
LLM
```

The model should see things like:

```
merchant: "Netflix"
amount: 22.99
category: "Entertainment"
month: "2026-08"
frequency: "monthly"
```

It should not need:

```
full account number
routing number
home address
customer ID
statement barcode
full legal name
bank account identifiers
raw PDF text
```

For the analytics explanation layer, the safest input is even more constrained:

```
Deterministic analytics output
  ↓
Sanitization
  ↓
LLM explanation
```

Example:

JSON

```
{
  "monthly_spending_change_pct": 18.4,
  "largest_growth_category": "Dining",
  "recurring_charges_total": 214.73,
  "top_merchants": [
    {"merchant": "Amazon", "total": 425.18}
  ]
}
```

Then the model can produce:

Dining was the largest contributor to your spending increase, while recurring monthly charges represented \$214.73.

The model does not need to calculate that number itself.

What should be stripped or masked

I'd put these in the sanitizer:

Full account numbers	→ REMOVE
Routing numbers	→ REMOVE
Full name	→ REMOVE
Address	→ REMOVE
Phone/email	→ REMOVE
Customer/member IDs	→ REMOVE
Check numbers	→ MASK or remove
Card numbers	→ REMOVE
Account last4	→ optionally pseudonymize

For example:

Chase Checking ••••1234

could become internally:

Account_A

before reaching the model.

That is useful because the model usually does not care whether the account ends in 1234.

One important nuance

Merchant names are not necessarily PII, so I would usually retain them.

```
Netflix
Starbucks
Amazon
Uber
```

Those are actually useful for explanations.

But descriptions can sometimes contain sensitive text:

```
TRANSFER TO JOHN SMITH
VENMO KERVINTZ NOEL
PAYROLL ACME CORP
CHECK #1838
```

So we should not blindly send `description_raw` to the model.

Use:

```
description_raw
    ↓
Merchant / entity normalization
    ↓
Sanitized description
```

Then:

```
VENMO JOHN SMITH
```

might become:

```
Venmo transfer
```

depending on the task.

Local model vs hosted model

You originally leaned toward a local LLM. That gives you a stronger privacy story.

Conceptually:

```
Sanitized structured data
    ↓
Local LLM
```

But I still would keep the sanitizer.

Why?

Because:

Privacy controls should not depend on where the model happens to run.

If you later switch from Ollama to a hosted API, the privacy boundary still works.

That gives us a strong invariant:

Model location is not the privacy control; sanitization is the privacy control.

And another:

The LLM receives the minimum data required for the specific task.

That is essentially data minimization.

So I'd lock the LLM boundary as:

```
Canonical / analytics data
    ↓
PII sanitizer
    ↓
Task-specific payload builder
    ↓
Local LLM
    ↓
Natural-language explanation
```

The next thing I'd design is **report generation and user-visible confidence/coverage**, because at that point we have the full path from upload to insights.

ok

Next is **report generation and user-visible confidence**.

At this point the system has validated statements, a unified ledger, deterministic analytics, and optional LLM explanations. The final layer should make two things obvious to the user:

1. **What the numbers say**
2. **How complete and trustworthy the analysis is**

I would structure the report like this:

Report

- └─ Coverage summary
- └─ Cash flow summary
- └─ Spending breakdown
- └─ Recurring expenses
- └─ Merchant analysis
- └─ Trends
- └─ Account summaries
- └─ Warnings / excluded statements
- └─ AI-generated explanation

The coverage section should come first, because a user should know whether the analysis is complete before trusting the totals.

Example:

Analysis coverage

Date range:

Jan 2025 - Dec 2026

Statements:

118 processed

2 failed

Accounts:

5 detected

Coverage status:

COMPLETED_WITH_WARNINGS

Then the report can say:

Your analysis excludes:

- Chase Checking - March 2025
- Capital One - November 2025

That is much better than burying failures somewhere in logs.

I would also separate **data confidence** from **AI confidence**.

For example:

Data quality

Statement coverage: 98.3%

Validated statements: 118 / 120

Possible duplicates: 3

Then the natural-language explanation can sit underneath that.

Another important rule:

The LLM explanation must never replace the underlying numbers.

If it says:

"Your transportation spending increased significantly."

The dashboard should also show:

```
Transportation
Previous period: $2,140
Current period:  $2,890
Change: +35.0%
```

That way the user can verify the explanation.

For v1, I would make the outputs:

```
Interactive dashboard
+
Financial summary report
+
CSV / JSON export
```

The CSV/JSON export matters because it makes the system less of a closed black box.

The user can inspect or reuse the cleaned data.

So the end of the architecture becomes:

```
Deterministic analytics
↓
Structured insights
↓
├── Dashboard
├── CSV / JSON export
└── Sanitized payload
    ↓
    Local LLM
    ↓
    Plain-English explanation
```

And I would lock these invariants:

- **Coverage and exclusions are always visible.**
- **AI explanations never replace source metrics.**
- **Users can export normalized data.**
- **Incomplete datasets must never be presented as complete analysis.**

At this point, we actually have the full logical architecture from upload to report.

The next useful step is not adding more components. It is to **walk the entire architecture from beginning to end and challenge every boundary before drawing the final diagram.**

ok

Good. Now we do the architecture review before drawing anything final.

Here is the full system as we have designed it so far:

```
USER
  ↓
Select one or more PDF statements
  ↓
File intake / upload
  ↓
<Upload successful?>
  ├ No → Mark failed → Retry / remove
  └ Yes
    ↓
  <Valid supported PDF?>
    ├ No → Reject + explain why
    └ Yes
      ↓
    Store PDF temporarily
      ↓
    Create statement job
      ↓
    Queue job reference + metadata
      ↓
    Worker retrieves statement
      ↓
    Inspect PDF
      ↓
    <Usable embedded text?>
      ├ Yes → Native text extraction
      └ No → OCR extraction
        ↓
    <Processing failure?>
```

- └ Retryable → Retry up to 2 times
- └ Non-retryable → Fail statement
- └ Success

↓

Extracted text

↓

Detect bank + statement format

↓

<Supported with sufficient confidence?>

- └ No → Unsupported / manual review
- └ Yes

↓

Select versioned bank parser

↓

Extract statement metadata + transactions

↓

Normalize into canonical schema

↓

Resolve account identity

↓

Financial consistency validation

↓

<Validation acceptable?>

- └ No → Flag / exclude from trusted analytics
- └ Yes

↓

Validated canonical statement

↓

Statement-level duplicate detection

↓

Transaction-level duplicate detection

↓

Unified ledger

↓

Merchant normalization

↓

Transaction categorization

↓

Deterministic analytics

↓

Structured financial insights

↓

PII sanitizer

↓

Task-specific LLM payload

↓

Local LLM

↓

Plain-English explanation

↓

Dashboard + report + CSV/JSON export

↓

Now the challenge pass.

1. Upload boundary — good

We correctly separated:

```
upload failure
≠
validation failure
```

That is clean.

2. Queue boundary — good

The queue stores:

```
job_id
batch_id
file_reference
metadata
```

not the PDF itself.

That is the right decision.

3. Temporary storage — one thing to enforce

We need a lifecycle rule:

```
Raw PDF exists only while required for processing.
```

After processing finishes, the system should delete it unless the user explicitly chooses otherwise.

That should become an invariant.

4. OCR decision — good

We made OCR a fallback:

```
native text first
OCR only when needed
```

Correct.

5. Bank detection — good, with one caution

We should not think only in terms of:

```
bank
```

We really care about:

```
bank + account type + statement layout/version
```

because the same bank can have multiple formats.

6. Canonical schema — essential and correct

This boundary is one of the strongest parts of the design:

```
bank-specific world
    ↓
canonical schema
    ↓
bank-independent world
```

Everything after normalization should stop caring which parser produced the data.

7. Validation — very strong

This is what makes the project more than a PDF parser.

We are checking:

```
Did we extract data?
```

and separately:

```
Does the financial data make sense?
```

That is a meaningful reliability decision.

8. Partial batch failure — correct

If:

```
117 succeed
3 fail
```

we still analyze the 117, but explicitly show incomplete coverage.

Good.

9. Deduplication — good, but keep it conservative

We already established the correct bias:

Missing a duplicate is less dangerous than deleting a real transaction.

So uncertain matches should stay in the ledger and get flagged.

10. Analytics — correct separation

This is critical:

```
code calculates  
LLM explains
```

Do not let that line blur later.

11. Categorization — good

Rule-based first, model fallback second.

Also:

```
user correction > automated categorization
```

Good invariant.

12. Privacy boundary — one thing I would change

Right now our flow says:

```
analytics  
↓  
PII sanitizer  
↓  
LLM
```

That is good for report explanation.

But categorization may also use the local model.

So the real rule should be:

Every path into a model must pass through a sanitizer and task-specific payload builder.

Meaning:

```
categorization candidate
```

```
↓
```

```
sanitize
```

```
↓
```

```
local model
```

and separately:

```
analytics result
```

```
↓
```

```
sanitize
```

```
↓
```

```
local model
```

The sanitizer is not one late-stage box. It is a **mandatory gateway in front of every model interaction**.

That is an important improvement.

13. One missing component: batch coordinator

We have individual statement jobs, but something needs to know:

```
How many statements belong to this batch?
```

```
How many finished?
```

```
How many failed?
```

```
When is the batch ready for aggregation?
```

So I would add:

```
Batch Coordinator
```

Conceptually:

```
Upload 120 statements
```

```
↓
```

```
Create batch
```

```
↓
```

```
Create 120 statement jobs
```

```
↓
```

```
Workers process independently
```

```
↓
```

```
Batch coordinator tracks:
```

```
117 completed
```

```
2 warnings
```

```
1 failed
```

Without this component, our background-processing story is incomplete.

14. Another missing boundary: persistence

We have not explicitly shown where these live:

```
batch metadata
job statuses
canonical transactions
validation results
analytics output
```

We do not need to choose PostgreSQL, SQLite, or anything else yet.

But architecturally, we need:

```
Application Data Store
```

because the system needs state across workers and across the user's session.

So our architecture probably needs two separate storage concepts:

```
Temporary File Storage
```

for raw PDFs

and:

```
Application Data Store
```

for normalized, non-raw system data.

That distinction is important.

15. Current architecture is now strong enough to diagram

At this point, I would lock in two new architectural components before drawing:

```
Batch Coordinator
Application Data Store
```

And one new invariant:

Every model call must pass through the privacy gateway.

Then we are ready to build the first real architecture diagram together—not a generic flowchart, but one that shows **user boundary, intake, queue, workers, storage, validation, aggregation, analytics, privacy gateway, and outputs**.

ok

Good. Now we switch from **architecture decisions** to **diagram construction**.

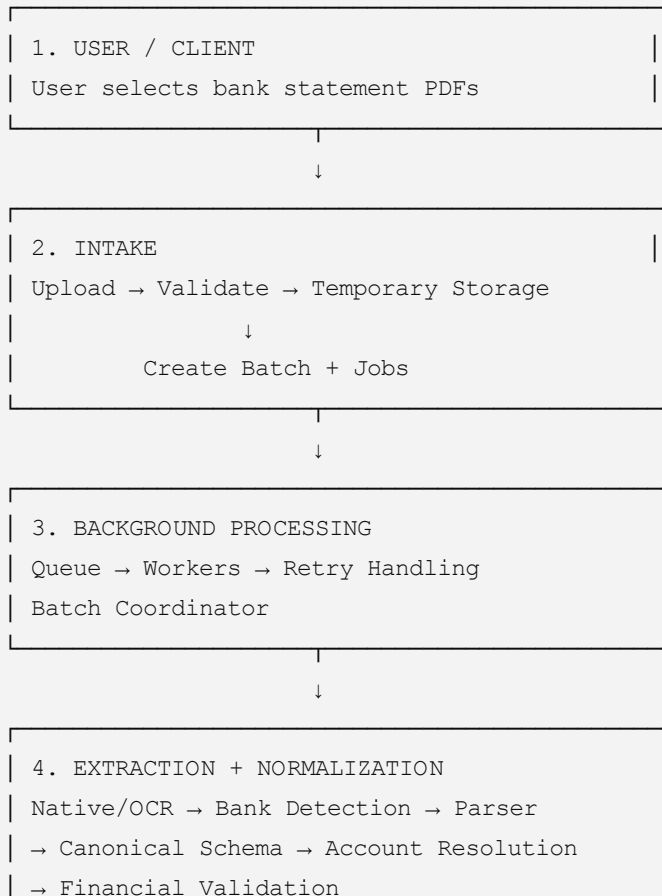
We should not draw every box at once. We'll build the diagram in layers.

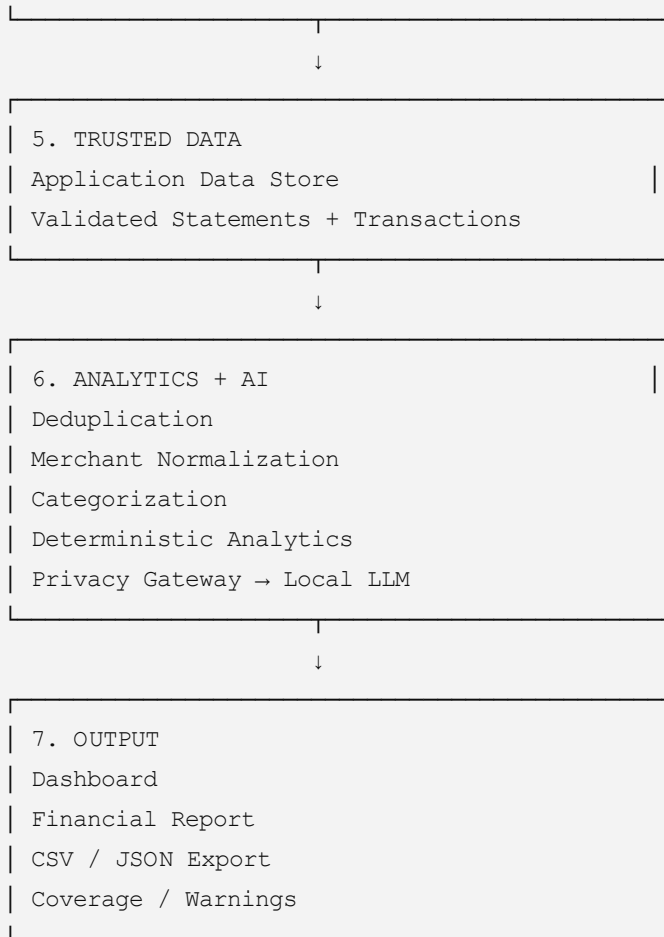
Diagram Step 1 — define the major zones

Your architecture naturally breaks into **7 zones**:

1. User / Client
2. Intake
3. Background Processing
4. Extraction + Normalization
5. Trusted Data
6. Analytics + AI
7. Output

Visually:





This is the **high-level skeleton**.

Why zones matter

Without zones, the diagram becomes a pile of boxes.

With zones, someone looking at it can immediately understand:

Where does user interaction happen?

Where does asynchronous work begin?

Where does raw sensitive data exist?

Where does trusted normalized data begin?

Where does the LLM sit?

Where are results produced?

That makes the diagram useful for a hiring manager, not just technically correct.

One important visual boundary

I would also draw a large **privacy boundary** around:

```
Temporary Storage
Background Processing
Extraction
Canonical Data
Analytics
Local LLM
```

with a label:

```
Sensitive financial data remains within
the controlled processing environment
```

And inside the AI section, another smaller boundary:

```
Privacy Gateway
      ↓
Local LLM
```

with:

Only sanitized task-specific data crosses this boundary.

So our first diagram decision is:

Use grouped architecture zones, not one giant programming flowchart.

The programming-flowchart symbols you found are still useful for the **decision points inside each zone**, but the overall artifact should be an architecture diagram with flowchart logic embedded inside it.

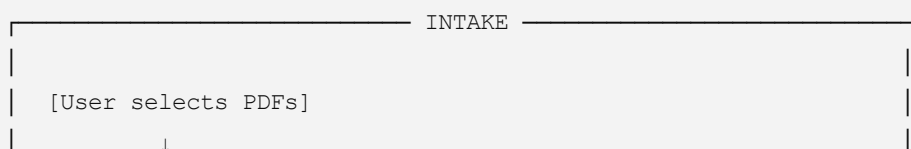
That hybrid is the right fit for this project.

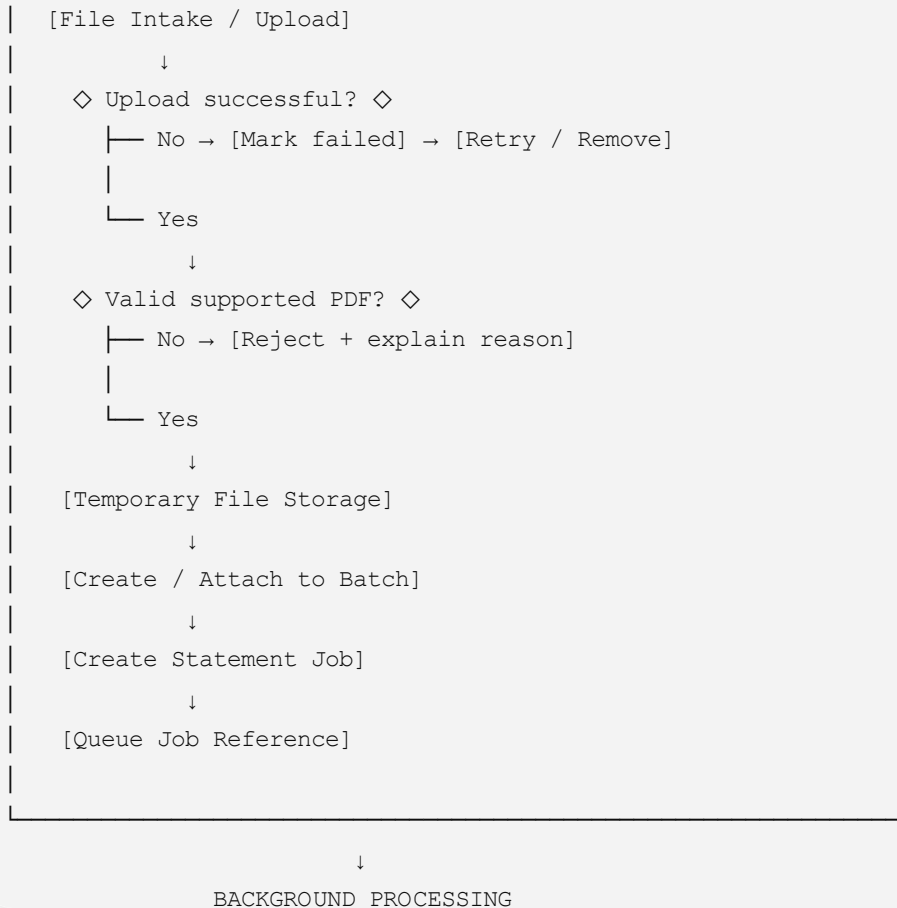
ok

Next we build **Diagram Step 2 — the Intake zone**.

This zone should be detailed enough to show decisions, but not so detailed that it overwhelms the architecture.

I would draw it like this:





Why I want **Batch** explicitly shown here

Because the batch is created from the user's upload session.

Imagine the user uploads 60 files:

```
Batch B001
├─ statement_job_001
├─ statement_job_002
├─ statement_job_003
├─ ...
└─ statement_job_060
```

The batch gives us the thing we eventually analyze as a whole.

Without it, we'd have 60 unrelated jobs and no clean way to answer:

```
How many finished?
Which failed?
Which belong to this analysis?
Is this analysis ready?
```

So the flow should really be:

```
User starts analysis
↓
```

```
Create Batch
```

```
↓
```

```
Process selected files
```

That slightly changes the intake layout.

A cleaner version is actually:

```
[User selects PDFs]
```

```
↓
```

```
[Create Analysis Batch]
```

```
↓
```

```
For each file
```

```
↓
```

```
[Upload / Intake]
```

```
↓
```

```
...
```

```
↓
```

```
[Create Statement Job]
```

```
↓
```

```
[Queue Job Reference]
```

That is better than creating the batch after storage.

One subtle thing: batch creation should not depend on every file succeeding

Suppose:

```
20 files selected
```

```
18 upload
```

```
2 fail
```

The batch still exists.

It should know:

```
selected = 20
```

```
uploaded = 18
```

```
upload_failed = 2
```

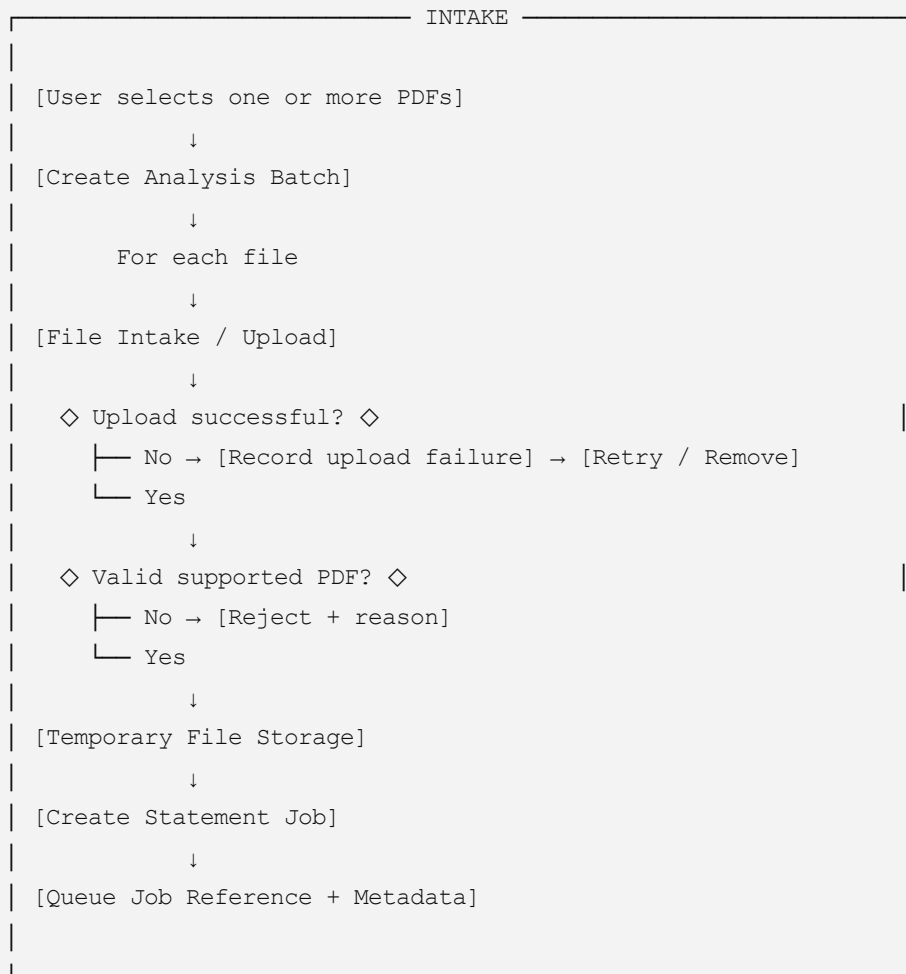
Later it might become:

```
processed = 17
```

```
processing_failed = 1
```

So the batch is the top-level tracking object from almost the beginning.

The final Intake zone I would lock in



This is the first zone I'd consider diagram-ready.

Next, we should build the **Background Processing zone**, where the queue, worker pool, retries, and batch coordinator interact.

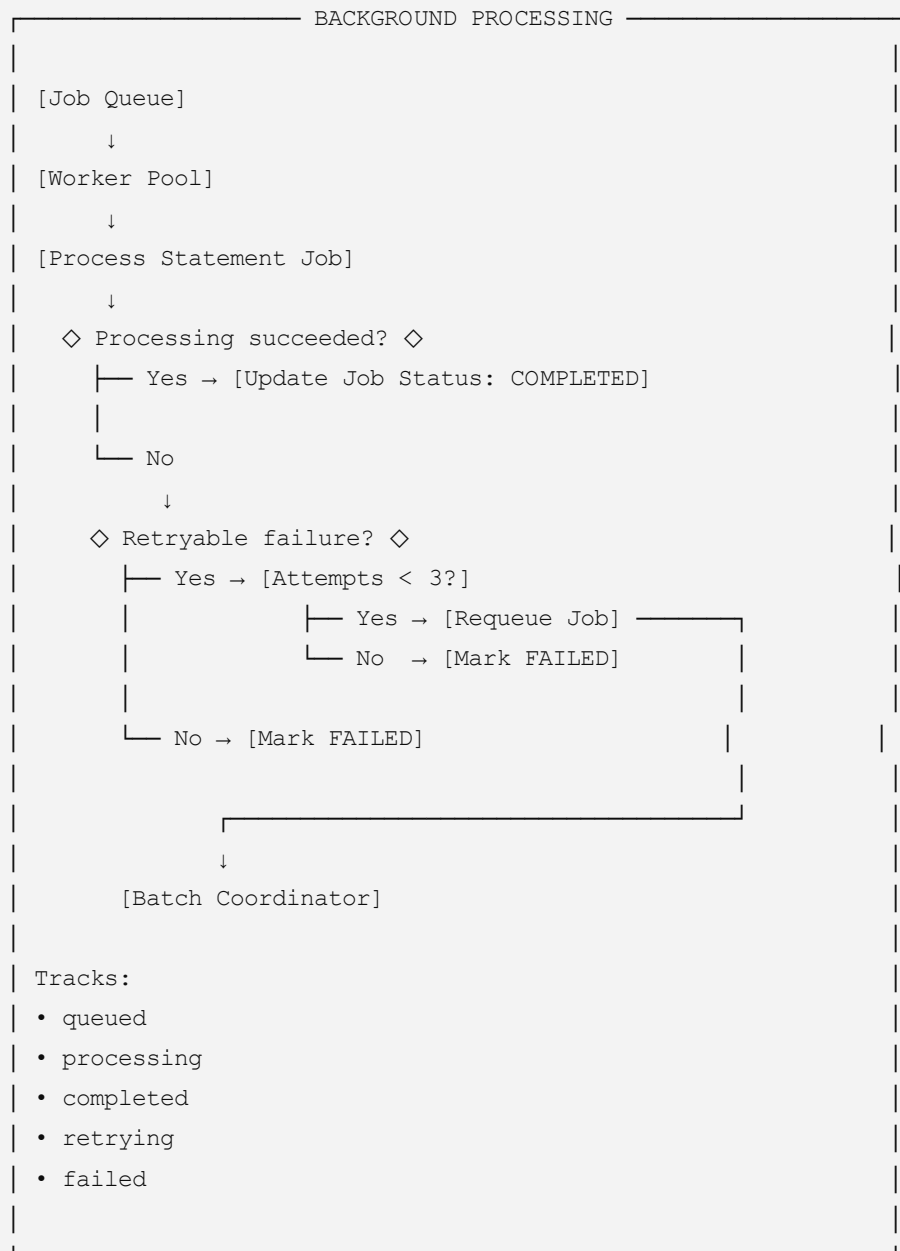
ok

Next is **Diagram Step 3 — Background Processing**.

This zone needs to answer four questions clearly:

1. Where does queued work wait?
2. Who processes it?
3. What happens on failure?
4. Who tracks the whole batch?

I'd structure it like this:



One correction though: the worker should **not mark the statement COMPLETED yet**.

Why? Because in this zone it has only successfully executed the processing job. We have not yet shown extraction, normalization, and financial validation.

So the better status would be something like:

```
PROCESSING
↓
EXTRACTED
↓
NORMALIZED
↓
VALIDATED
↓
COMPLETED
```

That means the worker owns execution, but the processing pipeline determines the actual job stage.

So the diagram should instead show:

```
[Worker receives job]
      ↓
[Execute statement processing pipeline]
      ↓
◇ Pipeline failure? ◇
```

Then the detailed extraction pipeline lives in the next zone.

The retry loop

We already decided:

First attempt + up to 2 retries = 3 total attempts.

So:

```
attempt_count = 1
      ↓
Failure
      ↓
Retryable?
  └─ No → FAILED
  └─ Yes
      ↓
    attempt_count < 3?
      └─ Yes → requeue
      └─ No  → FAILED
```

And importantly, requeueing should not mean duplicating the PDF. The same job points to the same temporary file reference.

Batch Coordinator

This component does not process PDFs.

Its job is orchestration:

```
Batch B001

selected: 120
uploaded: 119
queued:   14
processing: 6
completed: 96
```

```
warning:      2
failed:       1
```

Then it determines:

```
Are all processable statement jobs terminal?
      ↓
Yes
      ↓
Batch ready for aggregation
```

Where terminal means:

```
COMPLETED
FAILED
UNSUPPORTED
```

not:

```
QUEUED
PROCESSING
RETRYING
```

That gives us another useful invariant:

Aggregation begins only after every processable statement job has reached a terminal state.

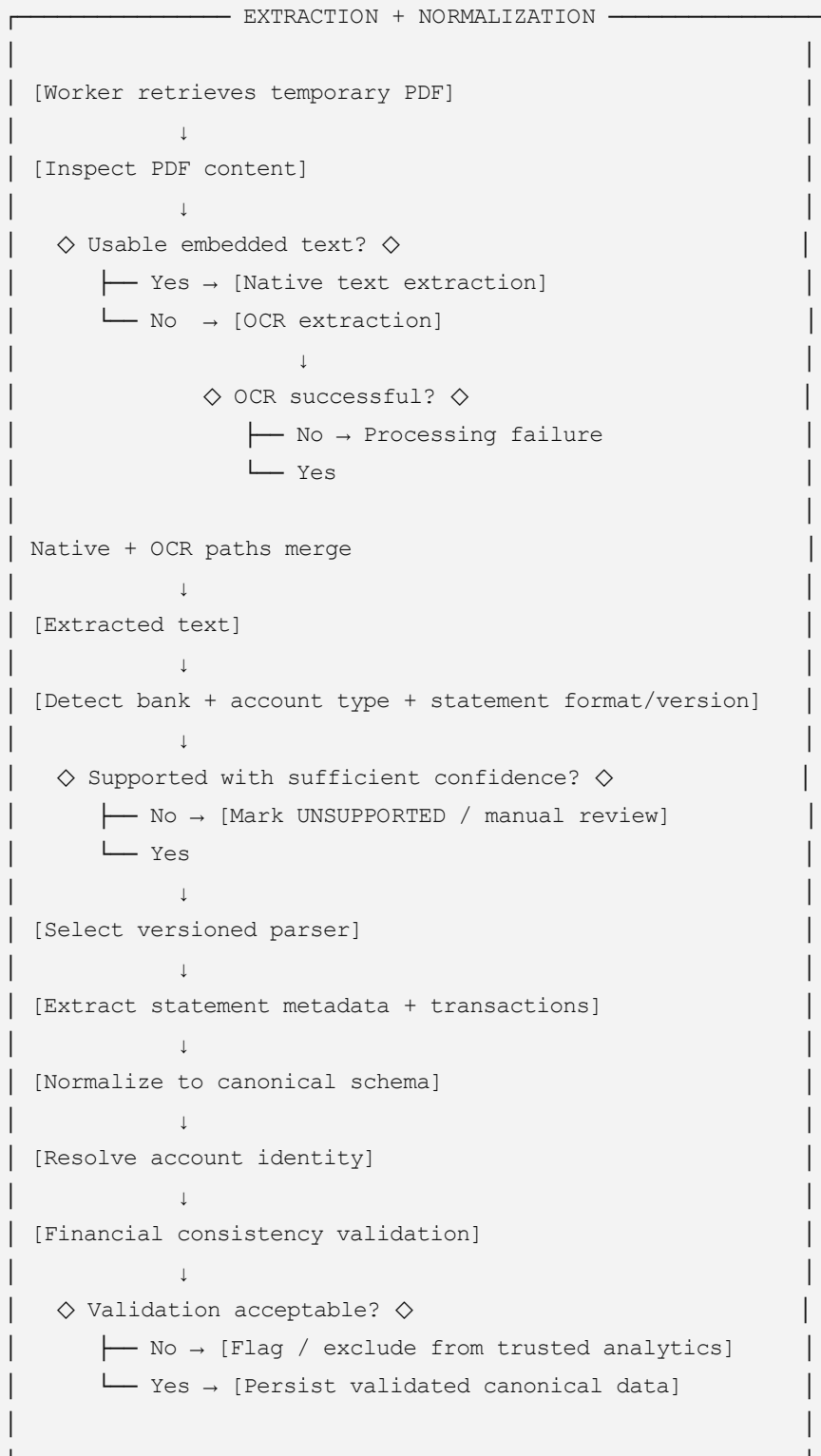
So I would lock the zone as:

```
Queue
  ↓
Worker Pool
  ↓
Statement Processing Pipeline
  ↓
Retry handling
  ↓
Job status updates
  ↓
Batch Coordinator
  ↓
Batch ready for aggregation
```

Next we should construct **Diagram Step 4 — Extraction + Normalization**, which will be the most detailed zone because that's where OCR, bank detection, parsers, canonicalization, account resolution, and validation all live.

Next is **Diagram Step 4 — Extraction + Normalization.**

This is the core processing zone. It should show the decision points we already agreed on without turning into spaghetti.



There are a few important design details behind this.

First, **native extraction** and **OCR** converge into the same contract:

```
Extracted text
```

Everything after that point should not care whether the text came from OCR or directly from the PDF.

Second, the detector should identify more than just the bank:

```
bank
+ account type
+ statement layout/version
```

That is what allows versioned parsers instead of one fragile parser per bank.

Third, the parser output should not leak downstream. It must be normalized before anything else consumes it:

```
Bank-specific parser output
      ↓
Canonical Statement
      +
Canonical Transactions[]
```

That boundary is crucial.

Finally, validation is the gate into **trusted data**. A statement can be successfully parsed and still fail financial reconciliation.

So the architectural rule for this zone is:

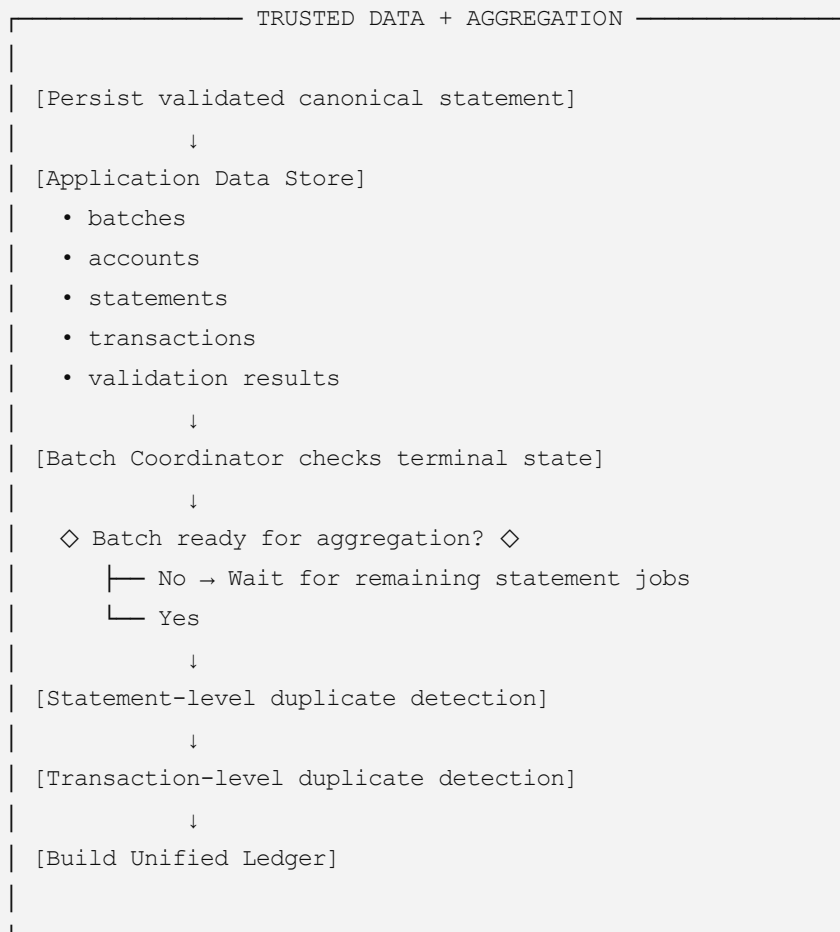
Parsing success does not equal trustworthy financial data. Validation is the trust gate.

After this, the next diagram zone is much simpler: **Trusted Data + Aggregation**, where validated statements enter the application data store and become the unified ledger.

ok

Next is **Diagram Step 5 — Trusted Data + Aggregation**.

This zone starts only after a statement has passed minimum validation.



The key idea here is that **the application data store is not raw-file storage**.

We now have two very different storage responsibilities:

Temporary File Storage

- raw PDFs
- short-lived
- sensitive
- delete after processing

Application Data Store

- normalized structured data
- persistent
- supports reports and analytics

That separation should be visible in the final architecture diagram.

For deduplication, we already agreed on two passes:

1. Statement-level deduplication
2. Transaction-level deduplication

Statement-level can use signals such as:

```
same account
same statement period
same opening balance
same closing balance
same transaction count
```

Transaction-level can use:

```
account
transaction date
posted date
timestamp if available
amount
direction
normalized description
reference ID if available
```

And the decision logic remains conservative:

```
HIGH CONFIDENCE
→ collapse duplicate

POSSIBLE DUPLICATE
→ keep + flag

NOT DUPLICATE
→ keep
```

Then we produce one important abstraction:

```
Unified Ledger
```

This is the point where the system stops thinking in terms of PDFs.

Before:

```
Chase_January.pdf
BOA_February.pdf
CapitalOne_March.pdf
```

After:

```
Unified Ledger
├─ Account A
|   ├─ transaction
|   ├─ transaction
|   └─ ...
```

└─ Account B
└─ Account C

That is a major architectural boundary.

So I would lock this invariant:

Analytics operates on the unified ledger, never directly on statements or PDFs.

Next is **Diagram Step 6 — Enrichment + Analytics + AI**, where merchant normalization, categorization, deterministic analytics, privacy gateway, and local LLM fit together.

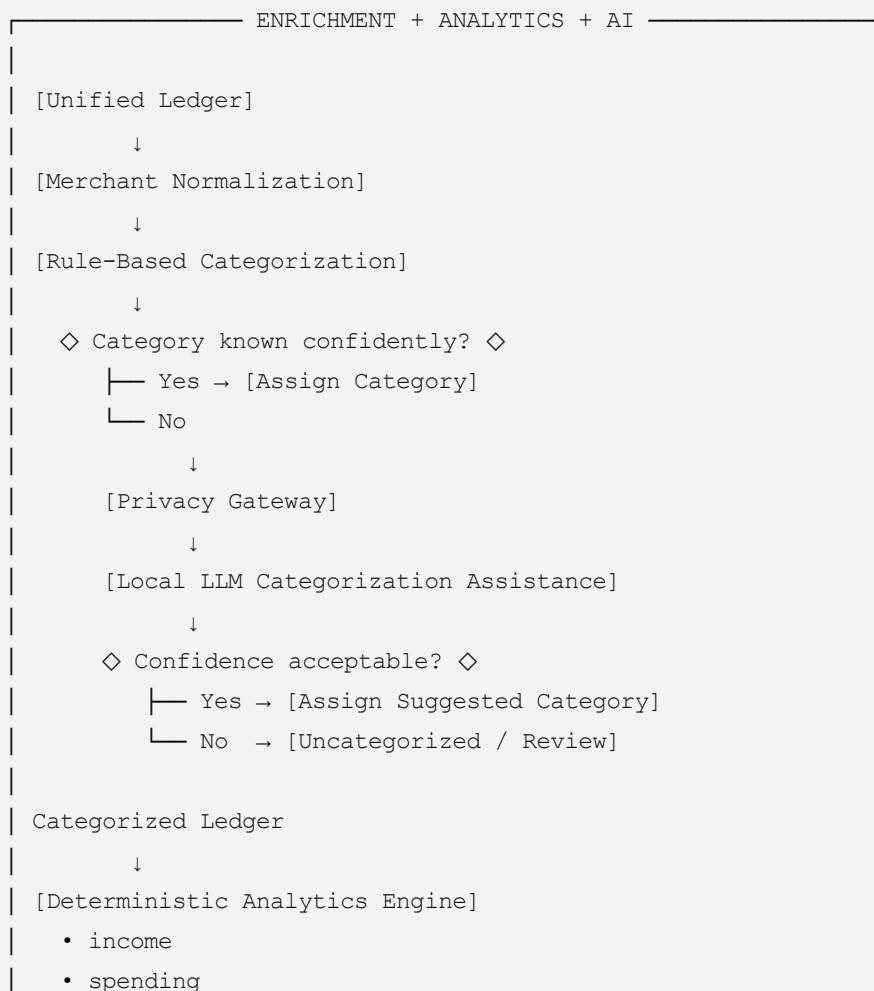
ok

Next is **Diagram Step 6 — Enrichment + Analytics + AI**.

This zone should make one thing visually obvious:

the deterministic system produces the financial facts, and the model only assists with uncertain categorization and natural-language explanation.

I'd draw it like this:



```

| • net cash flow
| • recurring charges
| • category totals
| • merchant totals
| • monthly trends
|
| ↓
| [Structured Financial Insights]
|
| ↓
| [Privacy Gateway]
|
| ↓
| [Task-Specific LLM Payload]
|
| ↓
| [Local LLM]
|
| ↓
| [Plain-English Explanation]
|

```

Why the Privacy Gateway appears twice

Conceptually, it is the same responsibility, but there are two model entry points:

```

Unknown transaction
    ↓
Privacy Gateway
    ↓
LLM categorization

```

and:

```

Analytics results
    ↓
Privacy Gateway
    ↓
LLM explanation

```

In the final architecture diagram, I would probably draw **one Privacy Gateway component** and route both model paths through it instead of duplicating the box.

Something like:

```

Unknown Tx  → [
Analytics  → | Privacy Gateway | → Local LLM
              ]

```

That visually enforces the rule:

There is no direct path to the model.

Merchant normalization comes before categorization

This matters because:

```
UBER *TRIP  
UBER TECHNOLOGIES  
UBER BV
```

should ideally become:

```
merchant = Uber
```

before category logic sees it.

Then:

```
Uber → Transportation
```

becomes much easier and more consistent.

Analytics engine stays deterministic

This box owns the numbers:

```
Monthly income  
Monthly expenses  
Net cash flow  
Recurring charges  
Merchant totals  
Category totals  
Trends
```

The model should never produce those values from scratch.

The model gets something more like:

```
Dining spending change: +28%  
Recurring charges total: $241/month  
Largest merchant increase: Amazon  
Coverage: 23/24 months
```

and turns that into useful language.

One more thing I would add here

A **User Corrections** path.

Because if the user changes:

XYZ MARKET
Shopping → Groceries

that correction should feed back into categorization rules.

So conceptually:

```
[User Category Correction]
      ↓
[Category Overrides / Merchant Mapping]
      ↓
Future Categorization
```

That makes the system improve without requiring model retraining.

So I'd lock these rules for this zone:

- **Merchant normalization happens before categorization.**
- **Deterministic rules are attempted before model assistance.**
- **Low-confidence categorization becomes `Uncategorized` , not a forced guess.**
- **All model access goes through the Privacy Gateway.**
- **The analytics engine owns the financial numbers.**
- **User corrections override automated categorization.**

Next is the final major zone: **Diagram Step 7 — Outputs + Coverage + Cleanup**, including the dashboard, report, exports, visible warnings, and deletion of temporary raw PDFs.

Today 12:12 AM

ok

Next is **Diagram Step 7 — Outputs + Coverage + Cleanup**.

This is the final zone, and it should do more than just show “Dashboard.” It needs to show that the system reports both the result **and the quality of the result**.

```
┌────────── OUTPUTS + COVERAGE + CLEANUP ─────────┐
│ [Deterministic Analytics Results]                    │
```


That directly supports one of the strongest decisions we made:

The system must never present incomplete analysis as complete.

The cleanup path should also be explicit because raw bank statements are the most sensitive artifacts in the system.

So the lifecycle becomes:

```
Upload raw PDF
  ↓
Temporary storage
  ↓
Processing
  ↓
Canonical data persisted
  ↓
Raw PDF no longer required
  ↓
DELETE
```

I would **not** automatically delete the normalized transaction data, because that is what powers the dashboard and report. The distinction remains:

```
Raw documents
→ temporary

Normalized application data
→ persistent
```

One final thing I'd include in the output zone is **traceability**. If a user clicks a transaction or questions a number, the system should retain enough metadata to say:

```
Transaction:
$142.19 — Electric Company

Source:
Account A
Statement: March 2026
Page: 3
Parser: bank_checking_v2
```

Even after the raw PDF is deleted, you can retain non-sensitive provenance metadata.

So the last invariant becomes:

Every reported financial fact should remain traceable to normalized source records and processing metadata, even after raw documents are deleted.

At this point, the architecture decisions are complete enough to draw the **first real end-to-end diagram**. The next step should be assembling all seven zones into one coherent diagram and then reviewing it for missing arrows, bad boundaries, or confusing responsibilities before styling it.

Today 12:13 AM

ok

Good. We now have enough decisions to assemble **Architecture Draft 1** as a real end-to-end design.

The order should be:

1. USER / CLIENT
2. INTAKE
3. BACKGROUND PROCESSING
4. EXTRACTION + NORMALIZATION
5. TRUSTED DATA + AGGREGATION
6. ENRICHMENT + ANALYTICS + AI
7. OUTPUTS + COVERAGE + CLEANUP

And the major cross-cutting elements are:

Temporary File Storage
Application Data Store
Batch Coordinator
Privacy Gateway
Retry Handling
Coverage / Warning State

Before I draw it, I'd lock one presentation decision:

The final diagram should be an architecture diagram first, with flowchart decision diamonds only where they add value.

So we should not use a decision diamond for every branch. Only for meaningful logic like:

Upload successful?
Valid supported PDF?
Usable embedded text?

Supported statement format?
Retryable failure?
Validation acceptable?
Category confidence acceptable?

Everything else should be architecture blocks and arrows.

That prevents it from looking like a giant algorithm flowchart.

The first full diagram should therefore show:

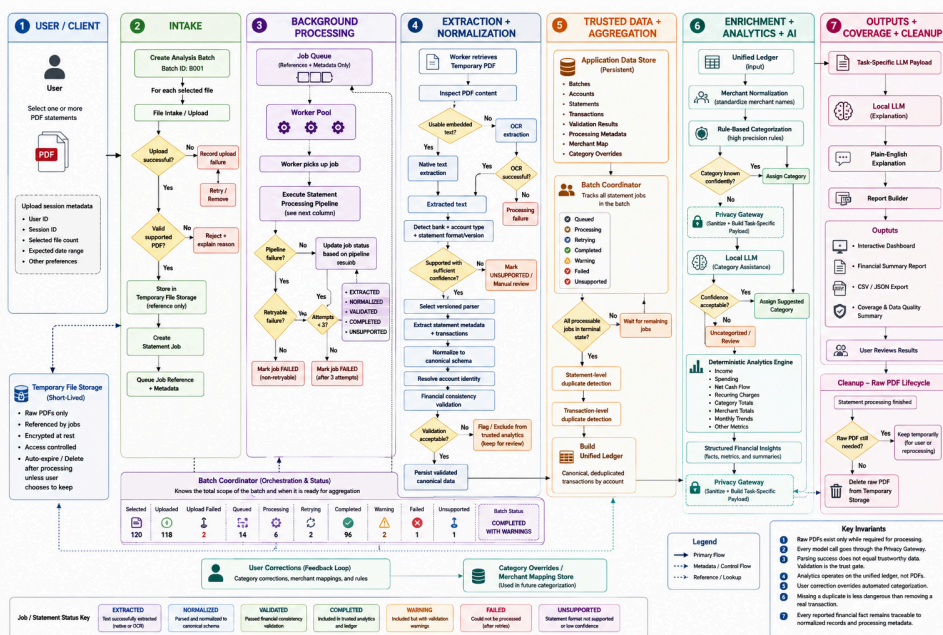
```
[User]
  ↓
[Intake]
  ↓
[Queue]
  ↓
[Workers]
  ↓
[Extraction Pipeline]
  ↓
[Canonical + Validated Data]
  ↓
[Unified Ledger]
  ↓
[Analytics]
  ↓
[Privacy Gateway]
  ↓
[Local LLM]
  ↓
[Dashboard / Report / Export]
```

with side connections to:

Temporary Storage
Application Data Store
Batch Coordinator

and the important failure/retry branches embedded inside the relevant zones.

That gives us the first diagram worth reviewing visually. The next move is to render **Draft 1** from exactly these decisions, then critique the diagram rather than changing the architecture blindly.



ChatGPT can make mistakes. Check important info.